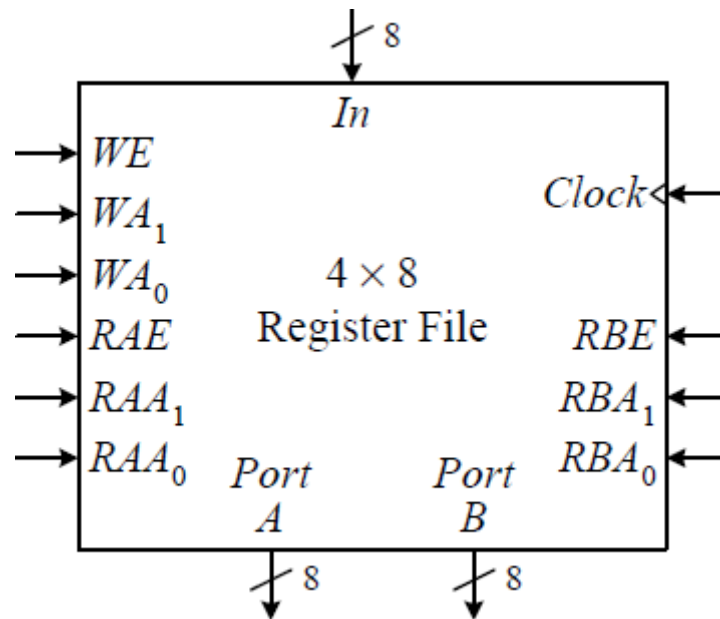


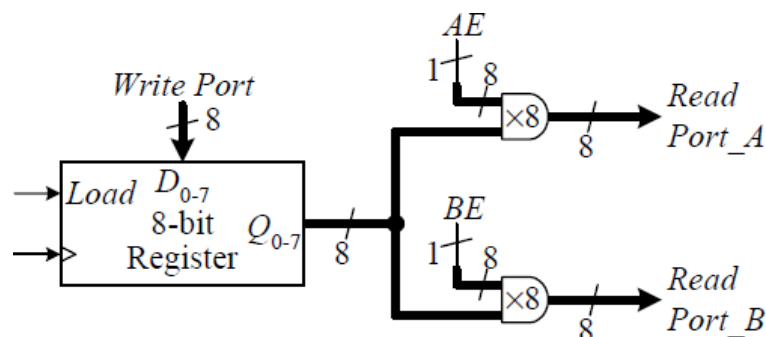
Register Files



Logic symbol for 4x8 register file

Learning objectives:

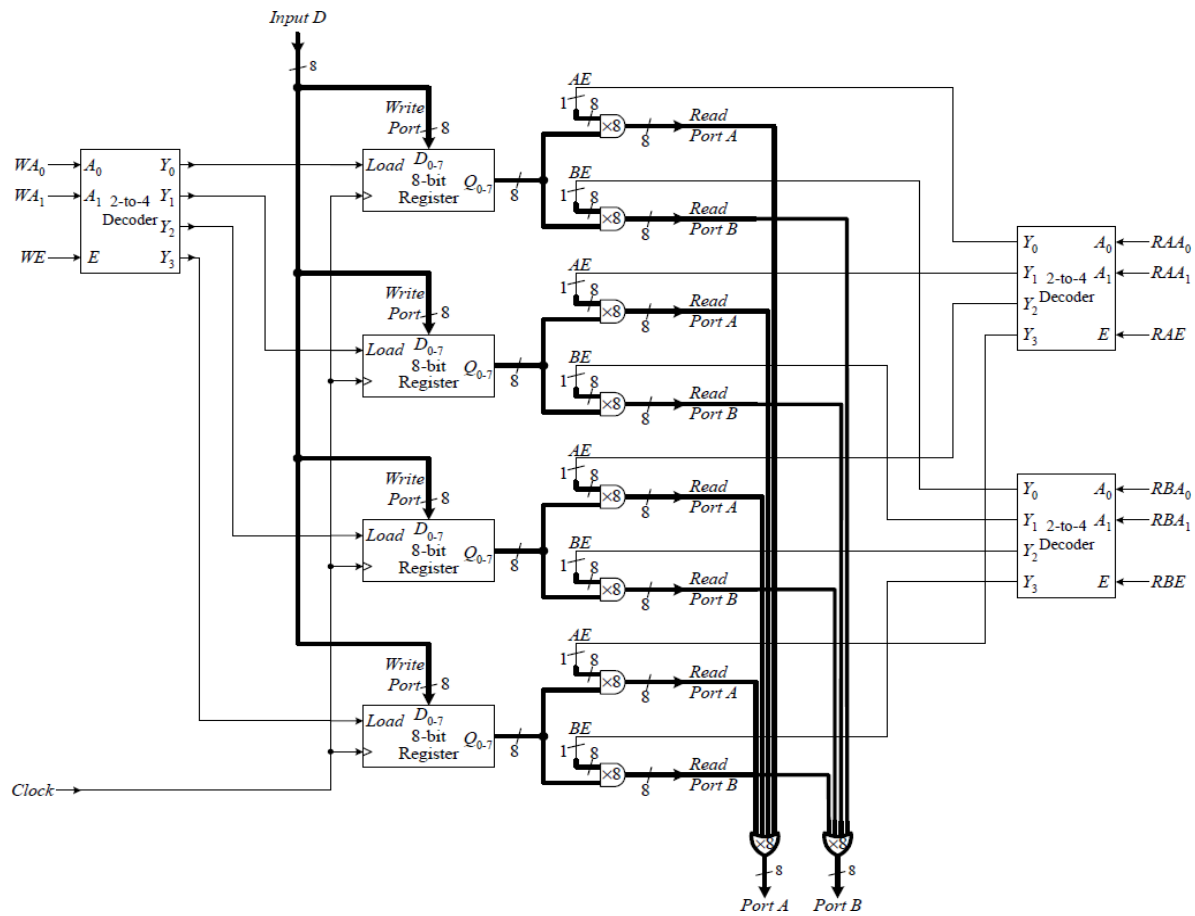
- To make control for reading of the data to the output port of the register file
- To control all the data output lines together
- Use AND gates or tri-state buffer to create a control signal for register file
- To design a fully functional register file to be used in other applications (e.g. ALU, design of microprocessor based systems, data processing logic)



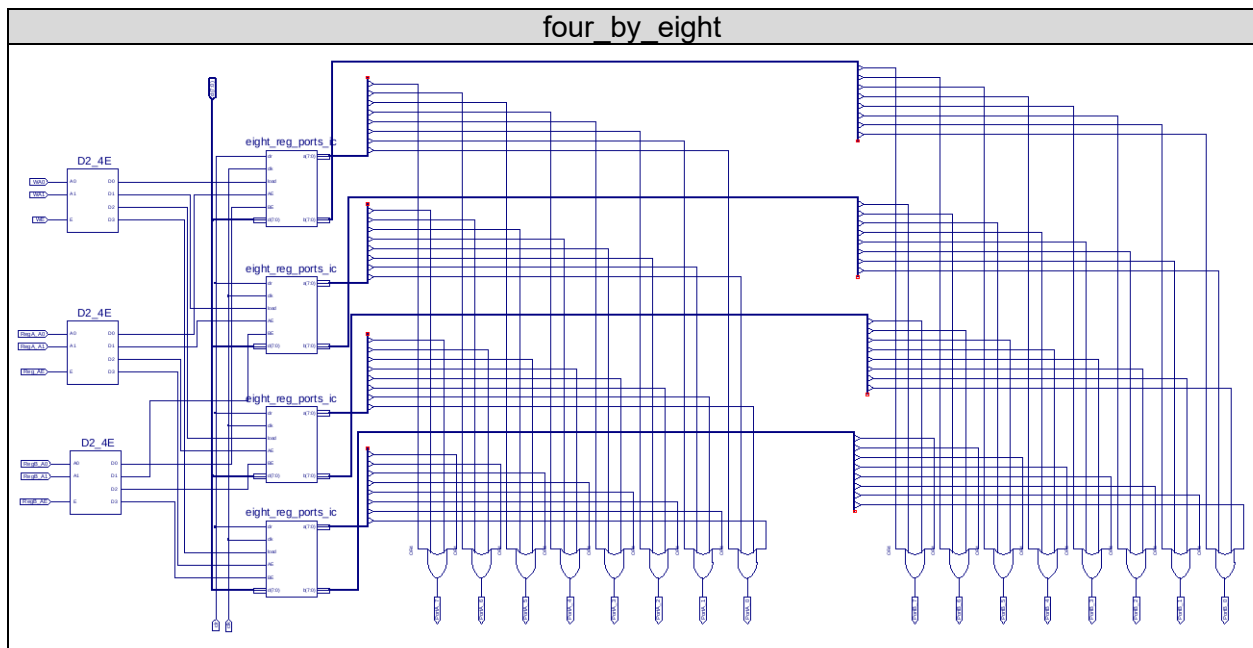
8-bit wide register file cell with one write port and two read ports

Exercises:

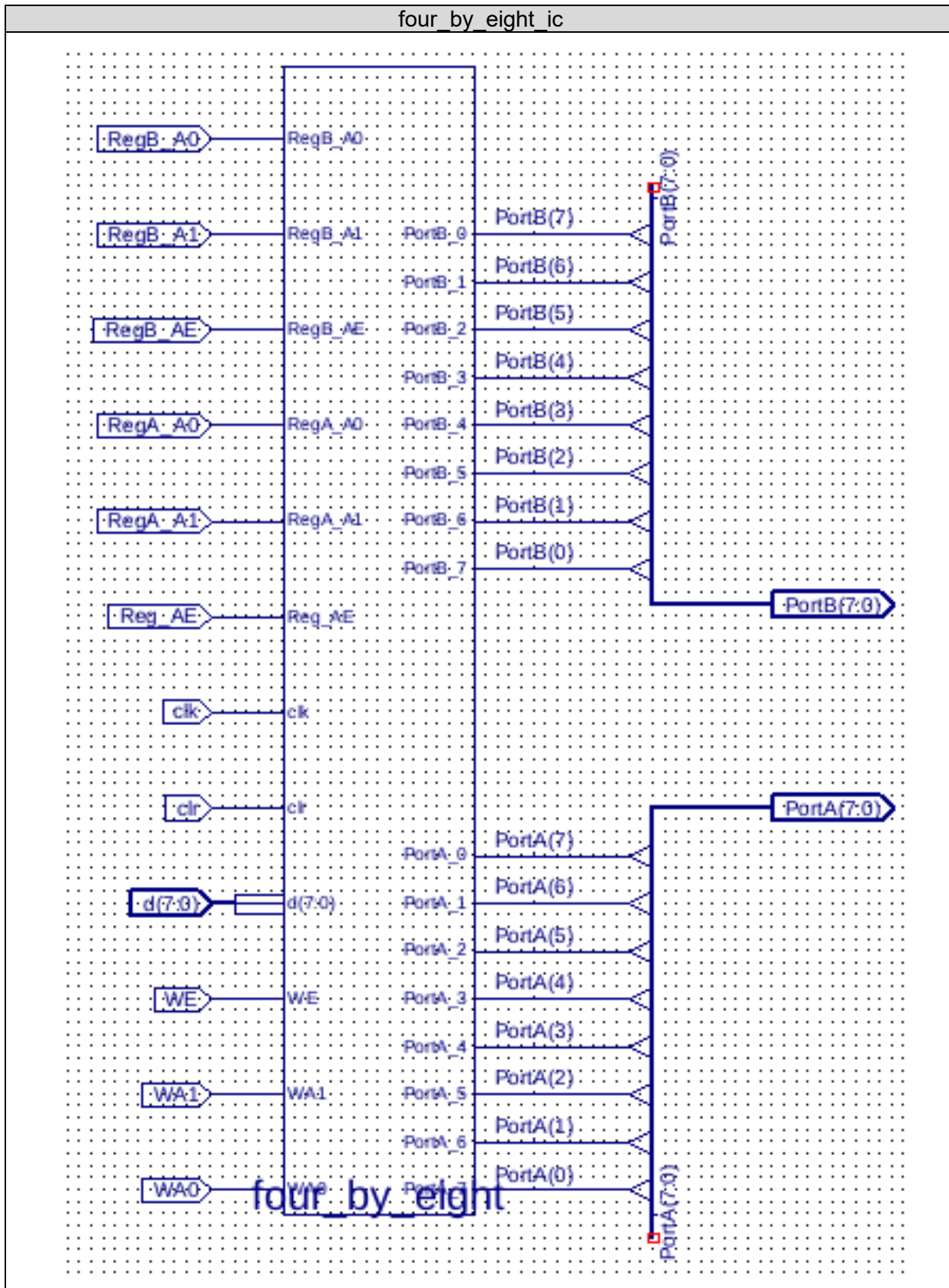
- Design of a 4x8 register file with one port and two read ports. Perform simulation using XILINX ISE



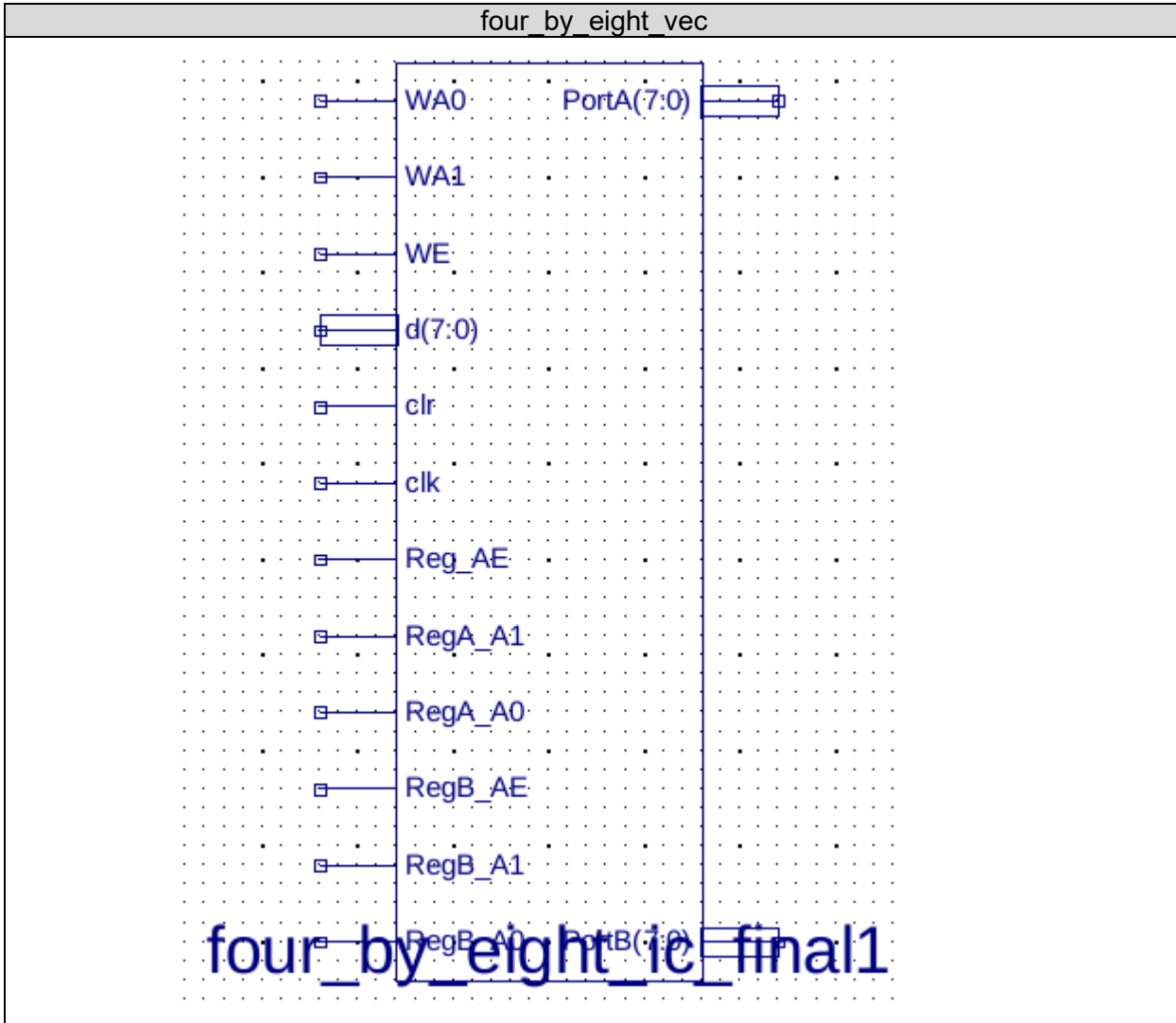
1. 4x8 Register File



2. 4x8 Register File IC



3. 4x8 Register File IC (Vectorized)



4. Test Bench

```

four_by_eight_tb
// Verilog test fixture created from schematic
/home/ise/four_eight_reg/eight_reg_ic.sch - Sun Jan 18 11:33:00 2026

`timescale 1ns / 1ps

module eight_reg_ic_eight_reg_ic_sch_tb();

// Inputs
reg clr;
reg clk;
reg load;
reg [7:0] d;

// Output
wire [7:0] q;

```

```

// Bidirs

// Initialize Inputs
initial
begin
    clr = 1'b1;
    load = 1'b0;
    d = 8'b0000_0000;
end

initial
begin
    #5 clr = 1'b0;
    load = 1'b1;

    #10 d = 8'b1110_0111;
end

initial
begin
    clk = 1'b0;
    forever
    #10 clk = ~clk;
end

// Instantiate the UUT
eight_reg_ic UUT (
    .clr(clr),
    .clk(clk),
    .load(load),
    .d(d),
    .q(q)
);

endmodule

```

5. Output Simulation

